Wykład 3

Pracownia Programowania

Pętla for. Funkcje.

Poniższe notatki są na podstawie materiałów Pani Dr. A. Zbrzeżny.

Instrukcja iteracyjna `for`

- Ogólna postać instrukcji `for` jest nastepująca:

```
for (init; test; modyfikacja) instrukcja
```

przy czym:

- wyrażenie `test` jest traktowane jako wyrażenie logiczne; jeżeli jest pominięte, to przyjmuje się wartość `1`;
- część `init` jest albo prostą deklaracją, albo jednym wyrażeniem, albo wyrażeniem przecinkowym, albo jest pominięta;
- część `modyfikacja` jest albo jednym wyrażeniem, albo wyrażeniem przecinkowym albo jest pominięta.
- nawet jeśli poszczególne elementy (`init`, `test` lub `modyfikacja`) są pominięte, nie wolno pominąć średników ani nawiasów okrągłych.

Instrukcja iteracyjna `for`

- Przebieg wykonania instrukcji `for` jest następujący:
  1. Jeżeli część `init` nie jest pominięta, to
     - jeżeli `init` jest wyrażeniem, to obliczana jest jego wartość;
     - jeżeli `init` jest prostą deklaracją, to zasięgiem zadeklarowanych w niej zmiennych jest cała pętla, czyli dalsza część `init`, `test`, `modyfikacja` oraz `instrukcja`.

     Część `init`, jeżeli nie jest pominięta, jest wykonywana zawsze (nawet gdy `instrukcja` nie będzie wykonana ani razu) i zawsze tylko raz – podczas „wejścia” do pętli.
  2. Obliczana jest wartość wyrażenia `test` lub przyjmowana jest wartość `1`, jeśli wyrażenie to jest pominięte. Jeśli wartością `test` jest `0`, wykonanie instrukcji iteracyjnej `for` kończy się.
  3. Wykonywane jest ciało instrukcji `for`, czyli instrukcja `instrukcja`.
  4. Jeśli `modyfikacja` nie jest pominięte, to obliczana jest jego wartość.
  5. Powrót do kroku 2.

Instrukcja iteracyjna `for`

- Instrukcja iteracyjna `for`

```
for (init; test; modyfikacja) instrukcja
```

jest równoważna następującej instrukcji:

```
{
    init;
    while (test) {
        instrukcja
        modyfikacja;
    }
}
```

- Wyjątkiem jest to, że instrukcja **`continue`** w ciele instrukcji `for` nie spowoduje pominięcia obliczenia wartości wyrażenia `modyfikacja` przed powrotem do obliczenia wartości wyrażenia `test`.

Przykład (Prosta instrukcja iteracyjna `for`)

```
#include <stdio.h>

int main(void)
{
    for (int j = 9; j >= 0; --j) {
        printf("%d ", j);
    }
    printf("\n");

    for (int j = 0; j < 10; ++j) {
        printf("%d ", j);
    }
    printf("\n");
}
```

Przykład (Zmienne zadeklarowane w części inicjalizacyjnej)

```
#include <stdio.h>

int main(void)
{
    for (int i = 0, j = 20; i < j; ++i, --j) {
        printf("(%d, %d)\n", i, j);
    }
    printf("\n");
    return 0;
}
```

Przykład (Część `init` może być wyrażeniem)

```
#include <stdio.h>

int main(void)
{
    int n = 0;
    for (printf("Loop start\n");
         printf("Loop test\n");
         printf("Iteration %d\n", ++n))
    {
        if (n > 5) {
            break;
        }
    }
    printf("\n");
}
```

Przykłady

- Program wykorzystujący typ **_Bool**
  `Bool.c`
- Program wykorzystujący plik nagłówkowy `stdbool.h` oraz makro **bool**
  `bool.c`

Plik nagłówkowy `stdbool.h`

- Po włączeniu do programu pliku nagłówkowego `stdbool.h` możemy deklarować zmienne typu **bool** zamiast **_Bool**.
- Jest to kwestia wyłącznie kosmetyczna, jednakże łatwiej pisać **bool** niż **_Bool**, a poza tym taka nazwa typu jest bliższa konwencji nazywania typów języka C: **`int`**, **`double`**, **`char`** itd.
- Ponadto, po włączeniu do programu pliku nagłówkowego `stdbool.h` możemy używać stałych **`true`** oraz **`false`**.

Przykład (Program wykorzystujący typ **bool**)

- `primes.c` – znajdowanie liczb pierwszych

Uwaga

Algorytm zastosowany w powyższym programie może być znacznie poprawiony w celu zwiększenia szybkości działania tego programu.

Podstawy języka C – instrukcja switch

Ogólna postać instrukcja switch jest następująca:

```
switch (wyrażenie) {
    case wartość_1:
        instrukcja
        ...
        break;
    ......
    ......
    case wartość_n:
        instrukcja
        ...
        break;
    default:
        instrukcja
        ...
        break;
}
```

instrukcja `switch`

- Ujęte w nawiasy `wyrażenie` jest porównywane kolejno z wartościami `wartość_1`, ..., `wartość_n` , które muszą być zwykłymi stałymi lub wyrażeniami stałymi.
- Jeżeli znaleziona zostanie wartość równa wartości wyrażenia, wykonywane są instrukcje występujące przy tej wartości, przy czym gdy instrukcji tych jest wiele, nie musimy ujmować ich w nawiasy klamrowe.
- Instrukcja **`break`** oznacza koniec danego przypadku i powoduje zakończenie wykonywania instrukcji **switch**.
- Trzeba ją wstawiać do każdego przypadku; w przeciwnym razie program będzie wykonywał instrukcje odpowiadające następnym wartościom.
- Specjalna fraza, **`default`**, jest używana, kiedy wyrażenie nie pasuje do żadnej z podanych wcześniej wartości.

Podstawy języka C – instrukcja switch

Ogólną postać instrukcji **switch** możemy wyrazić, korzystając z instrukcji **if**:

```
if (wyrażenie == wartość_1) {
    instrukcja
        ...
} else if (wyrażenie == wartość_2) {
    instrukcja
    ...
} else if (wyrażenie == wartość_3) {
...
} else if (wyrażenie == wartość_n) {
    instrukcja
    ...
} else {
    instrukcja
    ...
}
```

Funkcje

- Bardzo często zachodzi potrzeba wielokrotnego wykonania tej samej samej sekwencji instrukcji, ale z drobną różnicą, na przykład z inną wartością początkową.
- Funkcja to nazwana sekwencja instrukcji realizująca specyficzne zadanie.
- Celem definiowania funkcji jest podzielenie programu na fragmenty odzwierciedlające sposób myślenia o danym problemie.
- Funkcje stosuje się, aby uprościć program i zwiększyć jego czytelność.
- Funkcje ukrywają złożone obliczenia za pomocą jednej instrukcji – wywołania funkcji.
- Utworzenie funkcji pozwala zmniejszyć program poprzez eliminację powtarzającego się kodu.

Funkcje

- Składnia definicji funkcji jest następująca:

```
Typ nazwa(argumenty)
{
    ciągInstrukcji
}
```

- Pierwsza linia funkcji zaczyna się od nazwy typu, po które występuje nazwa funkcji oraz argumenty ujęte w nawiasy. Linia ta nazywa się nagłówkiem funkcji.
- Nazwa funkcji musi być identyfikatorem (ale nie może być słowem kluczowym).
- Lista argumentów może być pusta lub może składać się z dowolnej ilości argumentów oddzielonych przecinkami.

Funkcje

- Każda funkcja, w której pierwszym elementem nagłówka nie jest słowo kluczowe **void** musi zwrócić wartość typu Typ przy pomocy instrukcji o następującej składni:

```
    return 0;
```

- Wykonanie powyższej instrukcji powoduje obliczenia wartości wyrażenia wyrażenie, po czym wartość ta staje się wartością zwrotną danej funkcji.
- Wartość zwrotna może zostać zignorowana przez wywołującego funkcję; będzie ona wówczas odrzucona.

Przykład (Ciąg Collatza)

```
void ciag3nplus1(int n)
{
    while (n > 1) {
        printf("%d, ", n);
        if (n % 2 == 0) {
            n = n / 2;
        } else {
            n = 3 * n + 1;
        }
    }
    printf("%d.\n", n);
}
```

Przykład (Liczba cyfr w liczbie naturalnej)

```
int ile_cyfr(int n)
{
    int ile = 0;
    while (n > 0) {
        ile += 1;
        n = n / 10;
    }
    return ile;
}
```

Przykład (Liczba cyfr w liczbie naturalnej)

```
#include <stdio.h>
int main(void)
{
    int n;
    printf("Podaj liczbę naturalną dodatnią: ");
    scanf("%d", &n);
    if (n > 0) {
        printf("Liczba cyfr liczby %d: ", n);
        printf("%d\n", ile_cyfr(n));
    } else {
        printf("Skup się! Powiedziałem ");
        printf("\"Dodatnią\"!\n");
    }
}
```

Przykład (Niepoprawna kolejność)

```
#include <stdio.h>

int main(void)
{
    ......
        printf("%d\n", ile_cyfr(n));
    ......
}

int ile_cyfr(int n)
{
    ......
    ......
}
```

Przykład (Poprawna kolejność)

```
#include <stdio.h>

int ile_cyfr(int n)
{
    ......
    ......
}

int main(void)
{
    ......
        printf("%d\n", ile_cyfr(n));
    ......
}
```

Przykład (Poprawna i polecana kolejność)

```
#include <stdio.h>

int ile_cyfr(int n); // deklaracja funkcji

int main(void)
{
    ......
        printf("%d\n", ile_cyfr(n));
    ......
}

int ile_cyfr(int n)
{
    ......
}
```

Przykład (Algorytm Boyer'a i Moore'a wyszukiwania wzorca)

```
// awful.cc
#include<string.h>
int boyer_moore(const char *s,const char *t){int i,j,
shifts[256],n=strlen(s),m=strlen(t);for(j=0;j<256;++j)
shifts[j]=m;for(j=0;j<m;++j)shifts[(unsigned)t[j]]=m-j-1;
for(i=m-1,j=m-1;j>0;--i,--j)while(s[i]!=t[j]) {int
shift=shifts[(unsigned)s[i]];if(m-j>shift) i+=m-j;else
i+=shift;if(i>=n)return -1;j=m-1;}return i;}
```

Uwagi

- Łatwo sprawdzić, że powyższy plik się kompiluje:
  `gcc -c -std=c11 -Wall -Wextra -pedantic awful.c`
- Powstaje jednak wrażenie, że funkcja `boyer-moore` mogłaby być napisana nieco czytelniej ;-)

Przykład (Algorytm Boyer'a i Moore'a wyszukiwania wzorca)

```
int boyer_moore(const char *s,const char *t)
{
int i,j,shifts[256],n=strlen(s),m=strlen(t);
for(j=0;j<256;++j) shifts[j]=m;
for(j=0;j<m;++j)
shifts[(unsigned)t[j]]=m-j-1;
for(i=m-1,j=m-1;j>0;--i,--j)
while(s[i]!=t[j]) {
int shift=shifts[(unsigned)s[i]];
if(m-j>shift) i+=m-j;
else i+=shift;
if(i>=n)
return -1;
j=m-1;
}
return i;
}
```

Przykład (Algorytm Boyer'a i Moore'a wyszukiwania wzorca)

```
int boyer_moore(const char *s, const char *t)
{
    int i, j, shifts[256], n = strlen(s), m = strlen(t);
    for (j = 0; j < 256; ++j) shifts[j] = m;
    for (j = 0; j < m; ++j)
        shifts[(unsigned)t[j]] = m - j - 1;
    for (i = m - 1, j = m - 1; j > 0; --i, --j)
        while (s[i] != t[j]) {
            int shift = shifts[(unsigned)s[i]];
            if (m - j > shift)
                i += m - j;
            else
                i += shift;
            if (i >= n)
                return -1;
            j = m - 1;
        }
    return i;
}
```

Czytelność programów – program `astyle`

- Program `astyle` pozwala formatować programy napisane w językach `C`, `C++`, `C++/CLI`, `Objective-C`, `C#` i `Java`.
- Wywołanie programu `astyle` dla programu w języku `C` ma następującą postać: `astyle [opcje] prog.c`
- Jeżeli opcje nie występują, to program `astyle` dodaje jedynie wcięcia.
- Zamiast podawać opcje w linii poleceń, można je umieścić w pliku `/home/user/.astylerc`
- W zależności od podanych opcji `astyle` reformatuje kod źródłowy programu zwiększając istotnie jego czytelność.
- Dokumentacja programu znajduje się na stronie `http://astyle.sourceforge.net/astyle.html`

Przykład (Plik konfiguracyjny /home/user/.astylerc)

```
# Wybrane opcje dla programu astyle. Wywołanie: astyle program.cc
#
# One True Brace Style - jednolity styl dodawania nawiasów klamrowych
style=otbs
# Wielkość wcięcia 4 spacje
indent=spaces=4
# Zamień tabulatory na spacje
convert-tabs
# Wstaw spacje okalające dla operatorów
pad-oper
# Dodaj nawiasy klamrowe do jednoliniowych
# instrukcji warunkowych i iteracyjnych
add-brackets
# Usuń zbędne spacje wokół nawiasów
unpad-paren
# Wstaw spację między if, for, while a nawiasem
pad-header
# Wstaw puste pomiędzy kolejnymi głównymi blokami
break-blocks
```

Funkcje z pliku nagłówkowego `cctype` – c.d.

- `int isgraph(int c)` – Sprawdza czy znak przekazany jako argument jest znakiem graficznym.
- `int isprint(int c)` – Sprawdza czy znak przekazany jako argument jest znakiem drukowalnym.
- `int ispunct(int c)` – Sprawdza czy znak przekazany jako argument jest znakiem drukowalnym ale nie jest znakiem alfanumerycznym ani spacją.
- `int isspace(int c)` – Sprawdza czy znak przekazany jako argument jest białym znakiem.
- `int isxdigit(int c)` – Sprawdza czy znak przekazany jako argument jest cyfrą szesnastkową.

Funkcje z pliku nagłówkowego `cctype` – c.d.

- `int islower(int c)` – Sprawdza czy znak przekazany jako argument jest małą literą alfabetu.
- `int isupper(int c)` – Sprawdza czy znak przekazany jako argument jest dużą literą alfabetu.
- `int tolower(int c)` – Zwraca znak zamieniony z dużej litery na małą.
- `int toupper(int c)` – Zwraca znak zamieniony z małej litery na dużą.

Przykład (Niepolecany sposób)

```
#include <stdio.h>

int main(void)
{
    printf("Podaj pojedyńczy znak i wciśnij <ENTER>: ");
    char c; scanf("%c", &c);
    if ((c >= 'a'  &&  c <= 'z') ||
        (c >= 'A'  &&  c <= 'Z')) {
        printf("To jest litera.\n");
    } else if (c >= '0'  &&  c <= '9') {
        printf("To jest cyfra.\n");
    } else {
        printf("To jest znak specjalny.\n");
    }
}
```

Podstawy języka C testctype.c

Przykład (Funkcje z pliku nagłówkowego ctype)

```
#include <stdio.h>
#include <ctype.h>

int main(void)
{
    printf("Podaj pojedyńczy znak i wciśnij <ENTER>: ");
    char c; scanf("%c", &c);
    if (islower(c) || isupper(c)) {
        printf("To jest litera.\n");
    } else if (isdigit(c)) {
        printf("To jest cyfra.\n");
    } else {
        printf("To jest inny znak.\n");
    }
}
```

Podstawy języka C testgetchar.c

Przykład (Funkcja getchar)

```
#include <stdio.h>

// Funkcja getchar zwraca kod pierwszego znaku ze
// standardowego wejścia traktowany jako unsigned char
// przekształcony do typu int. W przypadku końca pliku
// lub błędu funkcja zwraca wartość EOF.

int main(void)
{
    int c = getchar();
    printf("Przeczytano znak o numerze %d.\n", c);

    return 0;
}
```

Przykład (Funkcja putchar)

```
#include <stdio.h>

// Funkcja putchar wysyła znak będący jej argumentem
// na standardowe wyjście.

int main(void)
{
    char c = 'a';
    while (c <= 'h') {
        putchar(c);
        ++c;
    }
    putchar('\n');
}
```

Podstawy języka C kopiuj.c

Przykład (Funkcje getchar i putchar)

```
#include <stdio.h>

// Kopiuje standardowe wejście na standardowe wyjście

int main(void)
{
    int c;
    while ((c = getchar()) != EOF) {
        putchar(c);
    }
    return 0;
}
```

Przekierowywanie standardowego wejścia i wyjścia

- Po skompilowaniu program z poprzedniego slajdu:
  `gcc -Wall -Wextra -pedantic kopiuj.c -o kopiuj`
  można go używać nie tylko do kopiowania standardowego wejścia na standardowe wyjście.
- Kopiowanie standardowego wejścia do pliku `output`:
  `./kopiuj > output`
- Kopiowanie pliku `input` na standardowe wyjście:
  `./kopiuj < input`
- Kopiowanie pliku `input` do pliku `output`:
  `./kopiuj < input > output`

Przykład (Kopiowanie z zamianą liter na wielkie litery)

```
#include <stdio.h>
#include <ctype.h>

// Kopiuje standardowe wejście na standardowe wyjście
// zamieniając małe litery na wielkie litery

int main(void)
{
    int c;
    while ((c = getchar()) != EOF) {
        putchar(toupper(c));
    }
    return 0;
}
```

Tablice

- Tablice są podstawową złożoną strukturą danych. Obecne są we wszystkich niemal językach programowania.
- Tablice są uporządkowanymi agregatami danych tego samego typu o wspólnym identyfikatorze (nazwie).
- Do poszczególnych elementów tablicy uzyskujemy dostęp poprzez ich numer kolejny (indeks) w tablicy.
- Numerowanie elementów zawsze rozpoczyna się od zera:
  - element pierwszy ma numer 0,
  - element ostatni ma indeks o jeden mniejszy od rozmiaru tablicy (czyli liczby elementów tablicy).
- Jeśli `tab` jest tablicą, to jej element o indeksie `k` jest oznaczany przez `tab[k]`.

Podstawy języka C tab-decl.c

Przykład (Deklarowanie tablic)

```
const int N = 20;

// tablica stu elementów typu short
short tab1[100];

// tablica dwudziestu elementów typu short
short tab2[N];

// tablica pięciu elementów typu short
short tab3[] = {10, 20, 30, 40, 50};

// tablica pięciu elementów typu short
// elementy o indeksach od 2 do 4 równe zero
short tab4[5] = {10, 20};

// tablica pięciu elementów; wszystkie elementy równe zero
short tab5[5] = {0};
```

Przykład (Operator `sizeof` dla tablic)

```
#include <stdio.h>

int main(void)
{
    int tab[100];
    printf("%zu %zu\n", sizeof(tab), sizeof(tab[0]));
}
```

Przykład (Przekazywanie tablic do funkcji)

```
void wczytaj(long a[], int n)
{
    for (int j = 0; j < n; ++j) {
        printf("tab[%d] = ", j);
        scanf("%ld", &a[j]);
    }
}
```

Uwagi

- Do funkcji przekazywany jest adres tablicy oraz liczba jej elementów.
- Elementy tablicy mogą być modyfikowane przez funkcję

Przykład (Przekazywanie tablic do funkcji)

```
void wypisz(const long a[], int n)
{
    for (int j = 0; j < n; ++j) {
        printf("%ld ", a[j]);
    }
    printf("\n");
}
```

Uwagi

- Do funkcji przekazywany jest adres tablicy oraz liczba jej elementów.
- Dzięki zastosowaniu słowa kluczowego `const` elementy tablicy nie mogą być modyfikowane przez funkcję.

Przykład (Przekazywanie tablic do funkcji)

```
#include <stdio.h>
void wypisz(const long a[], int n);
int main(void)
{
    long a[6] = {10, 20, 30};
    wypisz(a, sizeof(a) / sizeof(a[0]);
}

void wypisz(const long a[], int n)
{
    ......
}
```

Przykład (Program z wykorzystaniem tablic i funkcji)

- p04-10.c